

1. Data Collection

1.1 Overview

Data for this project were drawn from an existing corpus of publicly available Reddit posts compiled by Low et al. (2020). The dataset includes posts from both mental health-related subreddits (e.g., `r/depression`, `r/anxiety`) and control subreddits representing more general discussion topics.

Online platforms such as Reddit provide spaces where individuals may express personal experiences, including emotional distress, in relatively informal and semi-anonymous contexts. These environments offer an opportunity to examine how distress is communicated in natural language outside of clinical or structured settings.

This project focused on identifying and modeling expressions of emotional distress in Reddit posts using natural language processing techniques. While prior work has often relied on subreddit-level distinctions (e.g., mental health versus control communities), this project instead emphasized post-level labeling based on the content of each post.

All data were publicly available, and no personally identifying information was stored or used.

1.2 Sampling Strategy

Expressions of emotional distress were expected to vary across posts and were not uniformly distributed across subreddits. As a result, a structured sampling approach was used to ensure sufficient representation of both distress-related and non-distress content.

Posts were sampled from multiple subreddits included in the dataset, including:

- mental health-related subreddits (e.g., `depression`, `anxiety`, `addiction`), which were more likely to contain expressions of emotional distress
- control subreddits representing general Reddit activity, which provided baseline or non-distress content

Keyword-based filtering (e.g., “`depressed`,” “`struggling`,” “`burned out`”) was applied within subreddits to increase the likelihood of capturing distress-related posts. This filtering was used only to guide sampling and did not determine final labels.

This approach ensured that the labeled dataset included a range of posts, from likely distress-related content to more neutral or general discussion, while avoiding reliance on subreddit membership alone as a proxy for emotional state.

Because the goal of this project was to develop a model capable of detecting distress-related language, rather than estimating its prevalence, increasing the representation of distress-related posts was methodologically appropriate.

1.3 Data Scope

The dataset consisted of post-level Reddit data from multiple subreddits, including both mental health-focused communities and control communities.

From this dataset, **400 posts were sampled and manually labeled** to create a dataset for supervised learning.

The labeled subset was used for both model training and evaluation, including the creation of held-out test data.

Comments were not included in the initial modeling phase but may be incorporated in future work to examine how community members respond to posts expressing distress.

1.4 Data Splitting Strategy

The labeled dataset was divided into training and testing sets to evaluate model performance on unseen data. Approximately 70–80% of the labeled data were used for training, with the remaining 20–30% reserved as a held-out test set.

Because the dataset included posts drawn from both mental health and control subreddits, care was taken to ensure that both types of posts were represented in the training and test sets.

Model evaluation also incorporated cross-validation to provide a more stable estimate of performance given the relatively small dataset size.

1.5 Data Fields

For each post, the following fields were included:

- Post ID (created for this project)
- subreddit name
- date
- post text

The textual fields served as the primary inputs for classification. Metadata were retained mainly for data management, deduplication, and traceability during preprocessing and

manual labeling.

1.6 Data Collection Implementation

Data for this project were obtained from a previously compiled dataset of Reddit posts and did not require direct access to the Reddit API.

Posts were sampled from the dataset according to the sampling strategy described above, including both general and keyword-informed selection.

Because the dataset consisted of pre-collected posts, no additional data scraping or API-based collection was performed.

After sampling, posts were compiled into a working dataset for manual labeling and subsequent analysis.

```
In [1]: import pandas as pd
import os

data_dir = r"C:\Users\Eric's Desktop\OneDrive\Desktop\Data Science Masters Document

dfs = []

for file in os.listdir(data_dir):
    if file.endswith(".csv"):
        file_path = os.path.join(data_dir, file)

        print(f"Loading {file}...")

        df = pd.read_csv(file_path)

        # Keep only relevant columns
        df = df[["subreddit", "date", "post"]]

        dfs.append(df)

# Combine all into one DataFrame
all_data = pd.concat(dfs, ignore_index=True)

print("Combined dataset shape:", all_data.shape)
all_data.head()
```

Loading addiction_2018_features_tfidf_256.csv...
Loading addiction_2019_features_tfidf_256.csv...
Loading addiction_post_features_tfidf_256.csv...
Loading addiction_pre_features_tfidf_256.csv...
Loading adhd_2018_features_tfidf_256.csv...
Loading adhd_2019_features_tfidf_256.csv...
Loading adhd_post_features_tfidf_256.csv...
Loading adhd_pre_features_tfidf_256.csv...
Loading alcoholism_2018_features_tfidf_256.csv...
Loading alcoholism_2019_features_tfidf_256.csv...
Loading alcoholism_post_features_tfidf_256.csv...
Loading alcoholism_pre_features_tfidf_256.csv...
Loading anxiety_2018_features_tfidf_256.csv...
Loading anxiety_2019_features_tfidf_256.csv...
Loading anxiety_post_features_tfidf_256.csv...
Loading anxiety_pre_features_tfidf_256.csv...
Loading autism_2018_features_tfidf_256.csv...
Loading autism_2019_features_tfidf_256.csv...
Loading autism_post_features_tfidf_256.csv...
Loading autism_pre_features_tfidf_256.csv...
Loading bipolarreddit_2018_features_tfidf_256.csv...
Loading bipolarreddit_2019_features_tfidf_256.csv...
Loading bipolarreddit_post_features_tfidf_256.csv...
Loading bipolarreddit_pre_features_tfidf_256.csv...
Loading bpd_2018_features_tfidf_256.csv...
Loading bpd_2019_features_tfidf_256.csv...
Loading bpd_post_features_tfidf_256.csv...
Loading bpd_pre_features_tfidf_256.csv...
Loading conspiracy_2018_features_tfidf_256.csv...
Loading conspiracy_2019_features_tfidf_256.csv...
Loading conspiracy_post_features_tfidf_256.csv...
Loading conspiracy_pre_features_tfidf_256.csv...
Loading COVID19_support_post_features_tfidf_256.csv...
Loading depression_2018_features_tfidf_256.csv...
Loading depression_2019_features_tfidf_256.csv...
Loading depression_post_features_tfidf_256.csv...
Loading depression_pre_features_tfidf_256.csv...
Loading divorce_2018_features_tfidf_256.csv...
Loading divorce_2019_features_tfidf_256.csv...
Loading divorce_post_features_tfidf_256.csv...
Loading divorce_pre_features_tfidf_256.csv...
Loading EDAnonymous_2019_features_tfidf_256.csv...
Loading EDAnonymous_post_features_tfidf_256.csv...
Loading EDAnonymous_pre_features_tfidf_256.csv...
Loading fitness_2018_features_tfidf_256.csv...
Loading fitness_2019_features_tfidf_256.csv...
Loading fitness_post_features_tfidf_256.csv...
Loading fitness_pre_features_tfidf_256.csv...
Loading guns_2018_features_tfidf_256.csv...
Loading guns_2019_features_tfidf_256.csv...
Loading guns_post_features_tfidf_256.csv...
Loading guns_pre_features_tfidf_256.csv...
Loading healthanxiety_2018_features_tfidf_256.csv...
Loading healthanxiety_2019_features_tfidf_256.csv...
Loading healthanxiety_post_features_tfidf_256.csv...
Loading healthanxiety_pre_features_tfidf_256.csv...

```
Loading jokes_2018_features_tfidf_256.csv...
Loading jokes_2019_features_tfidf_256.csv...
Loading jokes_post_features_tfidf_256.csv...
Loading jokes_pre_features_tfidf_256.csv...
Loading legaladvice_2018_features_tfidf_256.csv...
Loading legaladvice_2019_features_tfidf_256.csv...
Loading legaladvice_post_features_tfidf_256.csv...
Loading legaladvice_pre_features_tfidf_256.csv...
Loading lonely_2018_features_tfidf_256.csv...
Loading lonely_2019_features_tfidf_256.csv...
Loading lonely_post_features_tfidf_256.csv...
Loading lonely_pre_features_tfidf_256.csv...
Loading meditation_2018_features_tfidf_256.csv...
Loading meditation_2019_features_tfidf_256.csv...
Loading meditation_post_features_tfidf_256.csv...
Loading meditation_pre_features_tfidf_256.csv...
Loading mentalhealth_2018_features_tfidf_256.csv...
Loading mentalhealth_2019_features_tfidf_256.csv...
Loading mentalhealth_post_features_tfidf_256.csv...
Loading mentalhealth_pre_features_tfidf_256.csv...
Loading parenting_2018_features_tfidf_256.csv...
Loading parenting_2019_features_tfidf_256.csv...
Loading parenting_post_features_tfidf_256.csv...
Loading parenting_pre_features_tfidf_256.csv...
Loading personalfinance_2018_features_tfidf_256.csv...
Loading personalfinance_2019_features_tfidf_256.csv...
Loading personalfinance_post_features_tfidf_256.csv...
Loading personalfinance_pre_features_tfidf_256.csv...
Loading ptsd_2018_features_tfidf_256.csv...
Loading ptsd_2019_features_tfidf_256.csv...
Loading ptsd_post_features_tfidf_256.csv...
Loading ptsd_pre_features_tfidf_256.csv...
Loading reddit_combined_dataset.csv...
Loading reddit_labeling_set.csv...
Loading relationships_2018_features_tfidf_256.csv...
Loading relationships_2019_features_tfidf_256.csv...
Loading relationships_post_features_tfidf_256.csv...
Loading relationships_pre_features_tfidf_256.csv...
Loading schizophrenia_2018_features_tfidf_256.csv...
Loading schizophrenia_2019_features_tfidf_256.csv...
Loading schizophrenia_post_features_tfidf_256.csv...
Loading schizophrenia_pre_features_tfidf_256.csv...
Loading socialanxiety_2018_features_tfidf_256.csv...
Loading socialanxiety_2019_features_tfidf_256.csv...
Loading socialanxiety_post_features_tfidf_256.csv...
Loading socialanxiety_pre_features_tfidf_256.csv...
Loading suicidewatch_2018_features_tfidf_256.csv...
Loading suicidewatch_2019_features_tfidf_256.csv...
Loading suicidewatch_post_features_tfidf_256.csv...
Loading suicidewatch_pre_features_tfidf_256.csv...
Loading teaching_2018_features_tfidf_256.csv...
Loading teaching_2019_features_tfidf_256.csv...
Loading teaching_post_features_tfidf_256.csv...
Loading teaching_pre_features_tfidf_256.csv...
Combined dataset shape: (1108700, 3)
```

Out[1]:

	subreddit	date	post
0	addiction	2018/01/01	Deciding to go of tramadol Well after never ta...
1	addiction	2018/01/02	My vyvanse addiction... It has gotten pretty b...
2	addiction	2018/01/02	Quitting coke and nicotine I'm gonna start by ...
3	addiction	2018/01/02	Is it OK to leave a drug addict you love? Man...
4	addiction	2018/01/02	My brother has a problem I'm not against weed....

In [2]:

```
##Clean text
# Drop missing or empty post text
all_data = all_data.dropna(subset=["post"])
all_data = all_data[all_data["post"].str.strip() != ""]

# Reset index
all_data = all_data.reset_index(drop=True)

print("Cleaned dataset shape:", all_data.shape)
all_data.head()
```

Cleaned dataset shape: (1108700, 3)

Out[2]:

	subreddit	date	post
0	addiction	2018/01/01	Deciding to go of tramadol Well after never ta...
1	addiction	2018/01/02	My vyvanse addiction... It has gotten pretty b...
2	addiction	2018/01/02	Quitting coke and nicotine I'm gonna start by ...
3	addiction	2018/01/02	Is it OK to leave a drug addict you love? Man...
4	addiction	2018/01/02	My brother has a problem I'm not against weed....

In [3]:

```
#inspect the dataset
all_data["subreddit"].value_counts().head(20)
```

```
Out[3]: subreddit
        legaladvice      164356
        personalfinance  128232
        depression       117592
        jokes            94570
        relationships     77315
        suicidewatch      66272
        anxiety           57810
        fitness           49881
        adhd              45705
        mentalhealth      45423
        parenting         33903
        conspiracy        29875
        bpd               24332
        lonely            23673
        socialanxiety     23037
        guns              22986
        meditation        16493
        EDAnonymous       14587
        divorce           12605
        autism            8879
        Name: count, dtype: int64
```

```
In [4]: #define keywords
keywords = [
    "depressed", "depression", "suicidal", "suicide",
    "hopeless", "struggling", "burned out", "burnt out",
    "anxious", "anxiety", "lonely", "isolated",
    "overwhelmed", "can't handle", "cannot handle",
    "mental health", "stressed", "stress",
    "miserable", "exhausted"
]

#keyword function
def contains_keyword(text, keyword_list):
    text = text.lower()
    return any(keyword in text for keyword in keyword_list)
```

```
In [5]: #create general and keyword samples
general_df = all_data.sample(n=500, random_state=42)
general_df["sampling_source"] = "general"

#create keyword sample
keyword_mask = all_data["post"].apply(
    lambda x: contains_keyword(x, keywords)
)

keyword_df = all_data[keyword_mask].copy()

# Sample from keyword matches
keyword_df = keyword_df.sample(n=500, random_state=42)
keyword_df["sampling_source"] = "keyword"

print("Keyword sample shape:", keyword_df.shape)
```

Keyword sample shape: (500, 4)

```
In [6]: #combine and deduplicate
all_posts = pd.concat([general_df, keyword_df], ignore_index=True)

# Remove duplicates based on text
all_posts = all_posts.drop_duplicates(subset="post").reset_index(drop=True)

print("Final dataset shape:", all_posts.shape)
```

Final dataset shape: (996, 4)

```
In [7]: #inspect
all_posts["sampling_source"].value_counts()
all_posts["subreddit"].value_counts()

pd.crosstab(all_posts["subreddit"], all_posts["sampling_source"])

all_posts[["post", "sampling_source"]].sample(10, random_state=42)
```

```
Out[7]:
```

	post	sampling_source
832	Still struggling tonight... Tonight just feels...	keyword
970	I don't know what it's like to be not depressive...	keyword
96	Memory problems are no joke Because you forget...	general
587	Worse anxiety attacks after starting Lexapro -...	keyword
450	I used to think I was indecisive... ... but no...	general
266	First job at 28, making me so anxious I starte...	general
290	California paycheck reversal I am about to get...	general
158	My (19F) boyfriend (20M) is acting really dist...	general
668	Does anyone here have a Whole Life policy that...	keyword
572	Threatened to be kicked out of school. Hi, I a...	keyword

```
In [8]: #save dataset
output_path = os.path.join(data_dir, "reddit_combined_dataset.csv")

all_posts.to_csv(output_path, index=False)

print("Saved file:", output_path)
print("Total posts:", len(all_posts))
```

Saved file: C:\Users\Eric's Desktop\OneDrive\Desktop\Data Science Masters Documents
 \Data 641 (applied language processing)\Project\data\reddit_combined_dataset.csv
 Total posts: 996

2. Labeling Set Creation

2.1 Purpose

A manually labeled subset of posts was created from the collected dataset to support supervised learning. Manual labeling was necessary because expressions of emotional distress are context-dependent and cannot be reliably identified using keywords alone. The labeled subset provided the reference data used for model training and evaluation.

2.2 Labeling Set Construction

Because the full collected dataset was larger than what could be labeled efficiently by hand, a smaller annotation set was drawn from the combined post pool. The final labeling set included **400 posts**, ensuring sufficient representation of both distress-related and non-distress content.

To increase the likelihood of capturing relevant examples while preserving variation in the dataset, the labeling set was constructed from both sampling sources:

- **General sample posts**, which reflect typical subreddit activity
- **Keyword-informed posts**, which are more likely to contain expressions of emotional distress

This approach supported the creation of a labeled dataset that included both likely positive cases and ordinary discussion content.

2.3 Label Definitions and Annotation Guidelines

Because emotional distress is context-dependent, manual annotation was required to create a reliable supervised learning dataset.

Label Definitions

Posts were manually labeled to indicate the presence or absence of emotional distress.

- **Label = 1 (Distress):**

The post expresses emotional distress, defined as experiences such as anxiety, sadness, overwhelm, loneliness, identity-related struggle, or difficulty coping. This included both explicit emotional language (e.g., "I feel depressed") and implicit indicators such as behavioral changes (e.g., withdrawal, inability to function) or maladaptive coping (e.g., self-harm).

- **Label = 0 (Non-distress):**

The post does not express emotional distress. This included neutral, informational, or problem-solving posts, as well as situations involving stress or difficulty without evidence of emotional strain or impaired coping.

Annotation Guidelines

Labeling decisions were guided by the following principles:

- **Distress vs. situational stress:**

Stressful situations alone (e.g., legal or financial issues) were labeled as non-distress unless accompanied by emotional overwhelm or difficulty coping.

- **Intensity and persistence:**

Mild or momentary emotions (e.g., "this is stressful") were labeled as non-distress, while stronger or persistent emotional expressions (e.g., "I'm overwhelmed," "I can't handle this") were labeled as distress.

- **Behavioral indicators:**

Posts indicating maladaptive coping (e.g., self-harm, withdrawal, inability to engage in daily activities) were labeled as distress even if emotional language was limited.

- **Past vs. present distress:**

References to past distress were labeled as non-distress if the post described recovery or resolution. Posts describing ongoing effects of past experiences were labeled as distress.

- **Tone vs. content:**

Casual or humorous tone did not override indicators of distress. Labels were based on underlying content rather than writing style.

Labeling Process

A total of **400 posts were manually labeled**. The dataset was constructed using a combination of:

- **General sampling**, representing typical Reddit activity
- **Keyword-informed sampling**, increasing the likelihood of capturing distress-related posts

The final labeled dataset included approximately **60% distress (1)** and **40% non-distress (0)** posts, providing a balanced representation for supervised learning.

2.4 Data Preparation for Labeling

The labeling file included the following fields:

- unique post ID (assigned within this project)
- subreddit name (hidden in Excel during manual labeling to reduce potential bias)
- post text
- sampling source (general or keyword, also hidden during manual labeling)

Additional empty columns were added for manual annotation, including a binary label indicating the presence or absence of emotional distress and a notes field for documenting ambiguous cases or labeling decisions.

The post text field represented the full textual content of each Reddit post and served as the primary input for classification.

The labeling file was exported in a spreadsheet-compatible format so that posts could be reviewed and coded manually.

```
In [9]: # Set a random seed for reproducibility
random_seed = 42

# Fix common encoding problems in the post text
def fix_encoding(text):
    if pd.isna(text):
        return text

    try:
        # First pass
        text = text.encode("latin1").decode("utf-8")
    except:
        pass

    try:
        # Second pass (for stubborn cases)
        text = text.encode("latin1").decode("utf-8")
    except:
        pass

    return text

# Apply encoding fix to the post column
all_posts = all_posts.copy()
all_posts["post"] = all_posts["post"].apply(fix_encoding)

# Clean text to fix stubborn errors
def clean_text(text):
    if pd.isna(text):
        return text

    replacements = {
        "â€™": "'",
        "â€œ": '"',
        "â€": '"',
        "â€“": "-",
        "â€”": "-",
        "â€¦": "...",
        "â€˜": "`",
        "Â": " ",
    }

    for bad, good in replacements.items():
        text = text.replace(bad, good)
```

```

        return text

all_posts["post"] = all_posts["post"].apply(clean_text)

# normalize whitespace for easier labeling
all_posts["post"] = (
    all_posts["post"]
    .str.replace("\r\n", "\n", regex=False)
    .str.replace("\r", "\n", regex=False)
    .str.replace(r"[ \t]+", " ", regex=True)
    .str.strip()
)

# Create a project-level post ID if it does not already exist
if "post_id" not in all_posts.columns:
    all_posts = all_posts.reset_index(drop=True).copy()
    all_posts["post_id"] = all_posts.index

# Remove author from the working dataset if present
all_posts = all_posts.drop(columns=["author"], errors="ignore")

# Define target labeling set size
target_n = 400
n_general = 200
n_keyword = 200

# Split the dataset by sampling source
general_posts = all_posts[all_posts["sampling_source"] == "general"].copy()
keyword_posts = all_posts[all_posts["sampling_source"] == "keyword"].copy()

# Sample posts from each source
general_label_set = general_posts.sample(
    n=min(n_general, len(general_posts)),
    random_state=random_seed
)

keyword_label_set = keyword_posts.sample(
    n=min(n_keyword, len(keyword_posts)),
    random_state=random_seed
)

# Combine the sampled posts into one labeling set
labeling_df = pd.concat(
    [general_label_set, keyword_label_set],
    ignore_index=True
)

# Shuffle the combined labeling set
labeling_df = labeling_df.sample(
    frac=1,
    random_state=random_seed
).reset_index(drop=True)

# Add empty columns for manual labeling
labeling_df["label"] = ""

```

```
labeling_df["label_notes"] = ""

# Select and order columns for the labeling spreadsheet
labeling_df = labeling_df[
    [
        "post_id",
        "subreddit",
        "date",
        "post",
        "sampling_source",
        "label",
        "label_notes"
    ]
]

# Inspect the result
print("Labeling set shape:", labeling_df.shape)
labeling_df.head()

# Save the labeling set for manual coding
labeling_df.to_csv("reddit_labeling_set.csv", index=False)

print("Saved file: reddit_labeling_set.csv")
```

Labeling set shape: (400, 7)

Saved file: reddit_labeling_set.csv

3. Modeling

3.1 Overview

After manual labeling was completed, the labeled dataset was prepared for supervised text classification. The goal of the modeling phase was to train a baseline model to distinguish between posts expressing emotional distress and posts not expressing emotional distress.

The modeling pipeline began with text preprocessing, followed by feature extraction using term frequency-inverse document frequency (TF-IDF). TF-IDF was used to convert the text of each post into a numeric representation that captures which words are important within a post relative to the full corpus.

A baseline classifier was then trained on these features to predict the binary distress label. Starting with a simple and interpretable model provided a useful benchmark before considering more complex approaches.

3.2 Text Preprocessing

Before feature extraction, the post text was normalized to improve consistency across documents. This preprocessing included:

- converting all text to lowercase

- replacing common character encoding artifacts
- removing non-ASCII characters
- collapsing extra whitespace

These steps were used to reduce noise in the text while preserving the meaning of the original posts.

3.3 Feature Extraction

The cleaned post text was transformed into TF-IDF features. This representation emphasizes words that are frequent within a post but less common across the dataset as a whole, making it well suited for text classification.

3.4 Baseline Modeling Approach

The first modeling approach used a TF-IDF representation of post text as input to a baseline classifier. This provided an initial measure of how well emotional distress could be detected from post language alone.

The labeled dataset was split into training and testing subsets to evaluate model performance on unseen data. Detailed evaluation metrics are presented in a later section.

```
In [10]: import pandas as pd
import re

# Load the labeled dataset

df = pd.read_csv("reddit_labeling_set_model.csv")

print(df.shape)
df.head()

# Check columns and label distribution
print(df.columns.tolist())
print(df["label"].value_counts(dropna=False))

# Keep only rows with valid labels
df = df[df["label"].isin([0, 1, "0", "1"])]

# Convert labels to integers
df["label"] = df["label"].astype(int)

print(df["label"].value_counts())

def normalize_text(text):
    text = str(text).lower()

    text = text.replace("â€™", "'")
    text = text.replace("â€œ", "'")
```

```

text = text.replace("â€", '')

text = re.sub(r"^\x00-\x7F+", " ", text)
text = re.sub(r"\s+", " ", text).strip()

return text

# Apply text normalization
df["clean_text"] = df["post"].apply(normalize_text)

df[["post", "clean_text"]].head()

```

```

(400, 6)
['post_id', 'subreddit', 'date', 'post', 'sampling_source', 'label']
label
1    240
0    160
Name: count, dtype: int64
label
1    240
0    160
Name: count, dtype: int64

```

Out[10]:

	post	clean_text
0	I'm so sick of people making you feel guilty f...	i'm so sick of people making you feel guilty f...
1	Do you have a (or several) âœœcomfort objectsâ...	do you have a (or several) "comfort objects" t...
2	Today I fucked an alien to death. It came in p...	today i fucked an alien to death. it came in p...
3	A Science Question Anyone know a lot about the...	a science question anyone know a lot about the...
4	In a dead end Hi, I just wanna write this to g...	in a dead end hi, i just wanna write this to g...

```

In [11]: #TF IDF features
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    stop_words="english",
    min_df=2,
    ngram_range=(1, 2)
)

X = tfidf.fit_transform(df["clean_text"])
y = df["label"]

print("TF-IDF shape:", X.shape)

```

TF-IDF shape: (400, 4279)

```

In [12]: #Train Test Split
from sklearn.model_selection import train_test_split

```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

print("\nTrain label distribution:")
print(y_train.value_counts(normalize=True))

print("\nTest label distribution:")
print(y_test.value_counts(normalize=True))
```

Train shape: (320, 4279)

Test shape: (80, 4279)

Train label distribution:

label

1 0.6

0 0.4

Name: proportion, dtype: float64

Test label distribution:

label

1 0.6

0 0.4

Name: proportion, dtype: float64

```
In [13]: # Model 1 - Logistic Regression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```


Accuracy: 0.7375

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.38	0.53	32
1	0.70	0.98	0.82	48
accuracy			0.74	80
macro avg	0.81	0.68	0.68	80
weighted avg	0.79	0.74	0.70	80

Logistic Regression Results

The baseline model achieved an overall accuracy of approximately 0.74. Performance differed notably across classes.

The model demonstrated **very strong recall for distress-related posts (recall = 0.98)**, indicating that it was highly effective at identifying posts containing potential distress signals. This suggests that the model is sensitive to language associated with emotional distress and rarely misses such cases.

However, performance for non-distress posts was substantially weaker. **Recall for non-distress posts was only 0.38**, meaning that many non-distress posts were incorrectly classified as distress. Despite high precision (0.92), the low recall indicates that the model struggled to correctly identify neutral or non-distress content.

This pattern reflects a tradeoff between **sensitivity (recall for distress)** and **specificity (correct identification of non-distress)**. The model appears biased toward predicting distress, likely due to both the class distribution (60% distress) and the presence of emotionally salient language in many posts.

Importantly, this behavior is consistent with the goal of the project. Because the objective is to detect expressions of emotional distress, prioritizing high recall for distress-related posts is desirable, even at the cost of increased false positives.

```
In [14]: # Model 2 - Linear SVM
from sklearn.svm import LinearSVC

svm_model = LinearSVC(random_state=42)
svm_model.fit(X_train, y_train)

y_pred_svm = svm_model.predict(X_test)

from sklearn.metrics import classification_report, accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_svm))
```

Accuracy: 0.825

Classification Report:

	precision	recall	f1-score	support
0	0.82	0.72	0.77	32
1	0.83	0.90	0.86	48
accuracy			0.82	80
macro avg	0.82	0.81	0.81	80
weighted avg	0.82	0.82	0.82	80

Linear SVM Results

The Linear SVM model achieved an overall accuracy of approximately 0.83, representing a clear improvement over the logistic regression baseline.

Performance was more balanced across both classes. For distress-related posts, the model maintained strong performance with a recall of 0.90 and an F1-score of 0.86, indicating that it remained highly effective at identifying posts expressing emotional distress.

Importantly, performance for non-distress posts improved substantially. Recall for non-distress posts increased to 0.72, compared to 0.38 in the logistic regression model. This indicates that the SVM was much better at correctly identifying neutral or non-distress content, reducing the number of false positives.

Overall, the Linear SVM achieved a better balance between sensitivity (detecting distress) and specificity (correctly identifying non-distress). While slightly less sensitive than the logistic regression model in detecting distress, it provided a more reliable and well-rounded classification performance.

These results suggest that Linear SVM is a strong candidate model for this task, as it maintains high detection of distress-related language while improving discrimination between distress and non-distress posts.

```
In [15]: #Model 3 - Neural Network (MLP)

from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(100,),
    max_iter=300,
    random_state=42
)

mlp.fit(X_train, y_train)

y_pred_mlp = mlp.predict(X_test)

from sklearn.metrics import classification_report, accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred_mlp))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_mlp))
```

Accuracy: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.47	0.60	32
1	0.73	0.94	0.82	48
accuracy			0.75	80
macro avg	0.78	0.70	0.71	80
weighted avg	0.77	0.75	0.73	80

Neural Network (MLP) Results

A simple neural network model (MLPClassifier) was evaluated as an additional modeling approach using TF-IDF features as input.

The model achieved an overall accuracy of approximately 0.75, which was comparable to the logistic regression baseline but lower than the Linear SVM model.

Performance for distress-related posts remained strong, with a recall of 0.94 and an F1-score of 0.82. This indicates that the neural network, like the logistic regression model, prioritized identifying distress-related content and maintained high sensitivity.

However, performance for non-distress posts was weaker, with a recall of 0.47. This suggests that the model continued to overclassify posts as distress, similar to the pattern observed in logistic regression.

Overall, the neural network did not outperform the Linear SVM model and did not substantially improve the balance between detecting distress and correctly identifying non-

distress posts. Given the relatively small dataset size and the effectiveness of linear models with TF-IDF features, these results suggest that more complex neural architectures may not provide significant additional benefit in this context.

```
In [16]: #tune SVM hyperparameters using GridSearchCV

from sklearn.model_selection import GridSearchCV
from sklearn.svm import LinearSVC

param_grid = {
    "C": [0.01, 0.1, 1, 10]
}

svm = LinearSVC(random_state=42)

grid_search = GridSearchCV(
    svm,
    param_grid,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best cross-validation score:", grid_search.best_score_)
```

```
Best parameters: {'C': 1}
Best cross-validation score: 0.8427691173086419
```

```
In [17]: best_svm = grid_search.best_estimator_

y_pred_svm_tuned = best_svm.predict(X_test)

from sklearn.metrics import classification_report, accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred_svm_tuned))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_svm_tuned))
```

```
Accuracy: 0.825
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.82	0.72	0.77	32
1	0.83	0.90	0.86	48
accuracy			0.82	80
macro avg	0.82	0.81	0.81	80
weighted avg	0.82	0.82	0.82	80

Hyperparameter Tuning

To optimize model performance, hyperparameters for the Linear SVM model were tuned using grid search with 5-fold cross-validation. The regularization parameter (C) was evaluated across multiple values, and model performance was assessed using F1-score.

The optimal parameter value was found to be $C = 1$, which corresponds to the default setting. Performance of the tuned model on the test set was identical to the baseline Linear SVM, with an accuracy of approximately 0.83 and balanced performance across both classes.

These results suggest that the baseline model was already well-specified and that additional hyperparameter tuning did not substantially improve performance. This indicates that model performance is likely driven more by the quality of the data and feature representation than by fine-tuning of model parameters.

4. Interpretation of Linguistic Features

While classification performance provides insight into how well the model distinguishes between distress and non-distress posts, it does not explain **how** those decisions are made. To better understand the model's behavior, feature-level analysis was conducted using the Linear SVM model.

Because the model is linear, each feature (i.e., each word or phrase in the TF-IDF representation) is assigned a coefficient that reflects its contribution to the model's predictions. Words with positive coefficients increase the likelihood that a post is classified as distress, while words with negative coefficients increase the likelihood of a non-distress classification.

Examining these feature weights allows for interpretation of the linguistic patterns associated with emotional distress. This analysis provides insight into the types of language that distinguish distress-related posts from more neutral or informational content, helping to connect model predictions back to meaningful human communication patterns.

```
In [18]: #Extract feature importance from SVM
import numpy as np

# Get feature names (words)
feature_names = tfidf.get_feature_names_out()

# Get model coefficients
coefficients = best_svm.coef_[0]

# Top words for distress (Label = 1)
top_distress_idx = np.argsort(coefficients)[-20:]
top_distress_words = feature_names[top_distress_idx]

# Top words for non-distress (Label = 0)
```

```

top_non_distress_idx = np.argsort(coefficients)[:20]
top_non_distress_words = feature_names[top_non_distress_idx]

print("Top distress words:\n", top_distress_words)
print("\nTop non-distress words:\n", top_non_distress_words)

```

Top distress words:

```

['head' 'tired' 'close' 'struggling' 'anymore' 've' 'depressed' 'feel'
'time' 'life' 'im' 'hate' 'friends' 'like' 'need' 'anxiety' 'don'
'suicide' 'feeling' 'just']

```

Top non-distress words:

```

['school' 'birthday' 'caffeine' 'thank' 'dating' 'alarm' 'girlfriend'
'probably' 'bad' 'adderall' 'son' 'https' 'invest' 'ptsd' 'boyfriend'
'meditation' 'subreddit' 'big' 'credit' 'company']

```

```

In [19]: #Make results easier to interpret
# Pair words with coefficients
word_coef_pairs = list(zip(feature_names, coefficients))

# Sort
sorted_words = sorted(word_coef_pairs, key=lambda x: x[1])

# Top non-distress
print("Top non-distress words:")
for word, coef in sorted_words[:20]:
    print(word, coef)

# Top distress
print("\nTop distress words:")
for word, coef in sorted_words[-20:]:
    print(word, coef)

```

Top non-distress words:

school	-0.8664463755044979
birthday	-0.6688716598856705
caffeine	-0.647178311038764
thank	-0.6416353787128493
dating	-0.6188416893557662
alarm	-0.6151062629723538
girlfriend	-0.6137066979117941
probably	-0.5988466101157849
bad	-0.5682183195779069
adderall	-0.5626280042084288
son	-0.5540915637486905
https	-0.5488209209768246
invest	-0.5461749641159612
ptsd	-0.5400241299073205
boyfriend	-0.5346747203705897
meditation	-0.5183508115399256
subreddit	-0.5063429944141832
big	-0.5046036314927707
credit	-0.4971202617791153
company	-0.4773449202303791

Top distress words:

head	0.6421754824094326
tired	0.6468273620750841
close	0.660198145954297
struggling	0.7061664975862997
anymore	0.7171862964691434
ve	0.7457813139677101
depressed	0.7602190757108559
feel	0.7685009579197363
time	0.8216828533206458
life	0.8416948218170955
im	0.876327429494714
hate	0.9306655282796295
friends	0.9367945991762369
like	0.9370958024474275
need	0.9394643592142969
anxiety	0.9761101935620422
don	1.0722685245006673
suicide	1.1383443178446413
feeling	1.1610403965304184
just	1.1803054491256566

```
In [20]: #Investigating the term "head"
#Find posts containing "head"
head_posts = df[df["clean_text"].str.contains(r"\bhead\b", na=False)]

# See a few examples
head_posts[["post", "label"]].sample(10, random_state=42)

#Separate by label
head_distress = head_posts[head_posts["label"] == 1]
head_non_distress = head_posts[head_posts["label"] == 0]

print("Distress examples:")
```

```
print(head_distress[["post"]].sample(5, random_state=42))

print("\nNon-distress examples:")
print(head_non_distress[["post"]].sample(5, random_state=42))
```

Distress examples:

	post
159	Navigating post-breakup feelings with an ex wh...
199	I feel useless, and I don't know what I should...
315	I hate that feeling where suicide is starting ...
309	Is this OCD? I need help. This is from an emai...
104	[TX] Charged with a misdemeanor A for weed and...

Non-distress examples:

	post
180	Chaotic mess with managers (Colorado, USA) So ...
368	Can I bring my DIL to small claims court for m...
197	I (24M) posted this to another sub, but I thou...
52	I [22M] am questioning how long my relationshi...
311	I don't want to get any bigger, but I don't kn...

4.2 Distress-Associated Language

The words most strongly associated with distress-related posts reflect a combination of **explicit emotional expressions**, **cognitive framing**, and **informal writing patterns**.

Several highly weighted terms correspond directly to emotional states, including *"depressed," "anxiety," "hate,"* and *"suicide."* These words indicate that many distress-related posts contain clear and explicit references to emotional suffering or crisis.

In addition to these direct indicators, many high-weight features reflect **subjective and internal language**, such as *"feel," "feeling," "life,"* and *"time."* These words suggest that distress-related posts often involve reflection on personal experiences, emotional states, and broader life circumstances rather than isolated external problems.

Other terms, such as *"struggling," "tired,"* and *"anymore,"* appear to capture **language of difficulty and exhaustion**, which may indicate ongoing or chronic distress. These words often occur in phrases that express a loss of coping ability (e.g., "I can't handle this anymore").

Interestingly, several high-weight features reflect **informal or conversational writing patterns**, including tokens such as *"im," "ve," "don,"* and *"just."* These may reflect the spontaneous and unstructured nature of distress-related posts, which are often written quickly and with less attention to formal grammar. These patterns may also indicate emotional immediacy or urgency.

Some high-weight features (e.g., *"head"*) were found to be less interpretable in isolation, as they appeared across both distress and non-distress contexts with different meanings. This highlights a limitation of word-level feature analysis, where individual terms may carry multiple meanings depending on context.

Overall, distress-related language appears to be characterized by a combination of explicit emotional terms, introspective framing, and informal expression, suggesting that individuals expressing distress tend to focus on internal experiences and communicate in a direct, personal manner.

4.3 Non-Distress Language

Words associated with non-distress posts tend to reflect **external, situational, or topic-specific content**, rather than internal emotional states.

Many of these terms relate to **practical or informational contexts**, such as *"credit," "company," "invest,"* and *"school."* These words suggest that non-distress posts are often focused on problem-solving, advice-seeking, or factual discussion.

Other terms reflect **specific life domains**, including relationships (*"girlfriend," "boyfriend"*), family (*"son"*), and everyday activities (*"birthday," "dating"*). While these topics may involve stress or conflict, they are typically discussed in a more situational or task-oriented manner rather than as expressions of emotional distress.

Some features, such as *"https"* and *"subreddit,"* appear to reflect structural or contextual artifacts of Reddit posts rather than meaningful linguistic signals. These likely indicate links or meta-discussion and are not directly related to emotional content.

Notably, the specific words associated with non-distress posts appear to reflect the topical structure of the subreddits included in the dataset. Many non-distress posts were drawn from communities focused on legal, financial, relational, or everyday life concerns (e.g., *legaladvice*, *personalfinance*, *relationships*). As a result, the language associated with non-distress classifications often reflects these domain-specific contexts rather than emotional expression.

This suggests that the model is, in part, distinguishing between distress and non-distress posts based on differences between **internally focused emotional language** and **externally focused, topic-driven discussion**.

Overall, non-distress language is characterized by **externally oriented, domain-specific, and informational language**, contrasting with the more introspective and emotionally expressive patterns observed in distress-related posts.

4.4 Interpretation

The contrast between distress and non-distress language suggests that the model distinguishes between posts based on the extent to which they emphasize **internal emotional experience versus external situations**.

Distress-related posts tend to include:

- explicit emotional vocabulary
- expressions of struggle or inability to cope
- informal and immediate language

In contrast, non-distress posts tend to include:

- domain-specific or situational vocabulary
- problem-solving or advice-oriented language
- less emphasis on internal emotional states

These findings suggest that the model is capturing meaningful differences in how emotional distress is expressed in natural language. More specifically, classification appears to be driven by whether a post is oriented toward **internal, subjective experience** or **external, situation-focused discussion**.

Overall, this supports the validity of the labeling approach and indicates that the model is identifying interpretable and linguistically coherent patterns in the data.

5. Conclusion

This project examined the use of natural language processing techniques to identify expressions of emotional distress in Reddit posts. A manually labeled dataset of 400 posts was constructed and used to train and evaluate multiple classification models, including logistic regression, support vector machines, and a simple neural network.

Among the models tested, the Linear SVM achieved the strongest overall performance, providing a balanced tradeoff between accurately identifying distress-related posts and correctly classifying non-distress content. Logistic regression demonstrated extremely high recall for distress but tended to overclassify posts as distress, while the neural network did not substantially improve performance over the baseline models.

Feature-level analysis provided insight into the linguistic patterns associated with distress. Distress-related posts were characterized by emotionally expressive, introspective, and informal language, while non-distress posts tended to reflect externally focused, topic-driven, and informational content. These findings suggest that the distinction between distress and non-distress is driven less by isolated keywords and more by broader patterns in how individuals describe internal experiences versus external situations.

Several limitations should be noted. Because posts were sampled from different subreddits, some distinctions between distress and non-distress may reflect differences in topic rather than purely linguistic expressions of emotional state. Additionally, the relatively small dataset size limits the complexity of models that can be effectively applied.

Future work could expand the dataset, incorporate contextual features such as comment responses, or explore more advanced language models to better capture nuanced expressions of distress. Despite these limitations, the results demonstrate that relatively simple models combined with carefully labeled data can effectively identify meaningful patterns in how emotional distress is expressed in online communities.

In []: